```cpp
/*
 * File:   main.cpp
 * Author: Silvia Vargas
 *
 * Created on 13 de noviembre de 2023, 10:23 AM
 */

#include <iostream>
#include <fstream>
#include <iomanip>

using namespace std;

#include "funciones.h"

int main(int argc, char** argv) {
    //crearLibrosBin();
    crearLibrosBin("libros.csv","libros.bin");
    mostrarLibrosBin("libros.bin","librosIni.txt");
    crearRegistroBin("RegistroDePrestamos.csv","RegistroDePrestamos.bin");
    mostrarRegistroBin("RegistroDePrestamos.bin","registroIni.txt");
    actualizar("libros.bin","RegistroDePrestamos.bin","ReportePrestamos.txt",
            "SolicitudesDePrestamos.csv");
    mostrarLibrosBin("libros.bin","librosAct.txt");
    mostrarRegistroBin("RegistroDePrestamos.bin","registroAct.txt");
    return 0;
}


/*
 * File:   Libros.h
 * Author: Silvia Vargas
 *
 * Created on 13 de noviembre de 2023, 10:24 AM
 */

#ifndef LIBROS_H
#define LIBROS_H

struct Libros{
    char codigo[8];
    char titulo[50];
    char autor[50];
    int aPublicacion;
    int cantidad;
    int prestamos;
};


#endif /* LIBROS_H */


/*
 * File:   LibroPrestado.h
 * Author: Silvia Vargas
 *
 * Created on 13 de noviembre de 2023, 10:26 AM
 */

#ifndef LIBROPRESTADO_H
#define LIBROPRESTADO_H

struct LibroPrestado{
    char codigo[8];
    int fecha;
};

#endif /* LIBROPRESTADO_H */

```

```
70
71  /*
72   * File:   RegistroDePrestamos.h
73   * Author: Silvia Vargas
74   *
75   * Created on 13 de noviembre de 2023, 10:27 AM
76   */
77
78  #ifndef REGISTRODEPRESTAMOS_H
79  #define REGISTRODEPRESTAMOS_H
80
81  struct RegistroDePrestamos{
82      int codigo;
83      char nombre[50];
84      struct LibroPrestado libro[7];
85      int cantidadDeLibros;
86  };
87
88  #endif /* REGISTRODEPRESTAMOS_H */
89
90
91  /*
92   * File:   funciones.h
93   * Author: Silvia Vargas
94   *
95   * Created on 13 de noviembre de 2023, 10:32 AM
96   */
97
98  #ifndef FUNCIONES_H
99  #define FUNCIONES_H
100
101 void crearLibrosBin(const char *,const char *);
102 void mostrarLibrosBin(const char *,const char *);
103 void crearRegistroBin(const char *,const char *);
104 void mostrarRegistroBin(const char *,const char *);
105 void actualizar(const char *,const char *, const char *,const char *);
106 int juntarFecha(int ,int ,int );
107 void procesarPrestamo(fstream &,fstream &,ofstream &,int , const char *,
108         const char *,int ,int );
109 int buscarLibro(fstream &, const char *);
110 int calcularNumRegistros(fstream &,int );
111 void escribirReporte(ofstream &,int , const char *,const char *,const char *,const char
    *);
112 int buscarPrestamo(fstream &,int );
113 void actualizarPrestamo(fstream &,fstream &,ofstream &,struct RegistroDePrestamos &,
114         int ,struct Libros &,int ,int );
115 bool verificarFechaDevoluciones(struct RegistroDePrestamos &,int );
116 #endif /* FUNCIONES_H */
117
118
119 /*
120  * File:   main.cpp
121  * Author: Silvia Vargas
122  *
123  * Created on 13 de noviembre de 2023, 10:23 AM
124  */
125
126 #include <iostream>
127 #include <fstream>
128 #include <iomanip>
129
130 using namespace std;
131
132 #include "funciones.h"
133
134 int main(int argc, char** argv) {
135     //crearLibrosBin();
136     crearLibrosBin("libros.csv","libros.bin");
137     mostrarLibrosBin("libros.bin","librosIni.txt");
```

```
    crearRegistroBin("RegistroDePrestamos.csv","RegistroDePrestamos.bin");
    mostrarRegistroBin("RegistroDePrestamos.bin","registroIni.txt");
    actualizar("libros.bin","RegistroDePrestamos.bin","ReportePrestamos.txt",
            "SolicitudesDePrestamos.csv");
    mostrarLibrosBin("libros.bin","librosAct.txt");
    mostrarRegistroBin("RegistroDePrestamos.bin","registroAct.txt");
    return 0;
}



/*
 * File:   funciones.cpp
 * Author: Silvia Vargas
 *
 * Created on 13 de noviembre de 2023, 10:32 AM
 */

#include <iostream>
#include <fstream>
#include <iomanip>

using namespace std;

#include<cstring>
#include "LibroPrestado.h"
#include "Libros.h"
#include "RegistroDePrestamos.h"

#include "funciones.h"

#define NO_ENCONTRADO -1


void crearLibrosBin(const char *nombArchCsv,const char *nombArchBin){
    ifstream archLibrosCsv(nombArchCsv,ios::in);
    if (not archLibrosCsv.is_open()){
        cout<<"Error: el archivo "<<nombArchCsv<<" no se pudo abrir"<<endl;
        exit(1);
    }
    ofstream archLibrosBin(nombArchBin,ios::out|ios::binary);
    if (not archLibrosBin.is_open()){
        cout<<"Error: el archivo "<<nombArchBin<<" no se pudo abrir"<<endl;
        exit(1);
    }
    struct Libros libroDat;
    char codigo[8],nombre[50],titulo[50],car;
    int anio,cant,prest;
    while(true){
        //archLibrosCsv.getline(codigo,8,',');
        archLibrosCsv.getline(libroDat.codigo,8,',');
        if (archLibrosCsv.eof()) break;
        archLibrosCsv.getline(libroDat.titulo,50,',');
        archLibrosCsv.getline(libroDat.autor,50,',');
        archLibrosCsv>>libroDat.aPublicacion>>car>>libroDat.cantidad>>car>>
                libroDat.prestamos;
        archLibrosCsv.get();
        archLibrosBin.write(reinterpret_cast<const char*>(&libroDat),
                sizeof(struct Libros));
    }
}

void mostrarLibrosBin(const char *nombArchBin,const char *nombArchRep){
    ofstream archReporte(nombArchRep,ios::out);
    if (not archReporte.is_open()){
        cout<<"Error: el archivo "<<nombArchRep<<" no se pudo abrir"<<endl;
        exit(1);
    }
    ifstream archLibrosBin(nombArchBin,ios::in|ios::binary);
```

```
207       if (not archLibrosBin.is_open()){
208           cout<<"Error: el archivo "<<nombArchBin<<" no se pudo abrir"<<endl;
209           exit(1);
210       }
211       struct Libros libroDat;
212       archReporte<<"LISTA DE LIBROS"<<endl;
213       archReporte<<"CODIGO"<<setw(30)<<"NOMBRE"<<setw(30)<<"AUTOR"<<setw(15)<<"ANIO"<<
214               setw(15)<<"CANTIDAD"<<setw(15)<<"PRESTAMOS"<<endl;
215       while(true){
216           archLibrosBin.read(reinterpret_cast<char *>(&libroDat),sizeof(struct Libros));
217           if (archLibrosBin.eof()) break;
218           archReporte<<left<<setw(10)<<libroDat.codigo<<setw(52)<<libroDat.titulo
219               <<setw(52)<<libroDat.autor<<setw(10)<<libroDat.aPublicacion<<setw(10)
220               <<libroDat.cantidad<<setw(10)<<libroDat.prestamos<<endl;
221       }
222   }
223
224   void crearRegistroBin(const char *nombArchCsv,const char *nombArchPrestBin){
225       ifstream archRegistroCsv(nombArchCsv,ios::in);
226       if (not archRegistroCsv.is_open()){
227           cout<<"Error: el archivo "<<nombArchCsv<<" no se pudo abrir"<<endl;
228           exit(1);
229       }
230       ofstream archPrestamosBin(nombArchPrestBin,ios::out|ios::binary);
231       if (not archPrestamosBin.is_open()){
232           cout<<"Error: el archivo "<<nombArchPrestBin<<" no se pudo abrir"<<endl;
233           exit(1);
234       }
235       struct RegistroDePrestamos registroDat;
236       int i,dd,mm,aa;
237       char car;
238       while(true){
239           archRegistroCsv>>registroDat.codigo;
240           if (archRegistroCsv.eof()) break;
241           archRegistroCsv.get();
242           archRegistroCsv.getline(registroDat.nombre,50,',');
243           registroDat.cantidadDeLibros=0;
244           i=registroDat.cantidadDeLibros;
245           while(true){
246               archRegistroCsv.getline(registroDat.libro[i].codigo,
247                       8,',');
248               archRegistroCsv>>dd>>car>>mm>>car>>aa;
249               registroDat.libro[i].fecha=aa*10000+mm*100+dd;
250               i++;
251               if (archRegistroCsv.get()=='\n') break;
252           }
253           registroDat.cantidadDeLibros=i;
254           archPrestamosBin.write(reinterpret_cast<const char *>(&registroDat),
255                   sizeof(struct RegistroDePrestamos));
256       }
257
258   }
259
260   void mostrarRegistroBin(const char *nombArchBin,const char *nombArchRep){
261       ofstream archReporte(nombArchRep,ios::out);
262       if (not archReporte.is_open()){
263           cout<<"Error: el archivo "<<nombArchRep<<" no se pudo abrir"<<endl;
264           exit(1);
265       }
266       ifstream archRegistroBin(nombArchBin,ios::in|ios::binary);
267       if (not archRegistroBin.is_open()){
268           cout<<"Error: el archivo "<<nombArchBin<<" no se pudo abrir"<<endl;
269           exit(1);
270       }
271       struct RegistroDePrestamos registroDat;
272       archReporte<<"LISTA DE PRESTAMOS"<<endl;
273       while(true){
274           archRegistroBin.read(reinterpret_cast<char *>(&registroDat),
275                   sizeof(struct RegistroDePrestamos));
```

```
276         if (archRegistroBin.eof()) break;
277         archReporte<<"Usuario:"<<setw(10)<<registroDat.codigo<<' '<<left<<
278                 setw(50)<<registroDat.nombre<<"Cantidad: "<<setw(3)<<
279                 registroDat.cantidadDeLibros<<endl<<right;
280         for (int i=0;i<registroDat.cantidadDeLibros;i++)
281             archReporte<<setw(10)<<registroDat.libro[i].codigo<<setw(10)<<
282                     registroDat.libro[i].fecha<<endl;
283 
284     }
285 }
286 
287 void actualizar(const char *nombLibrosBin,const char *nombRegBin,
288         const char *nomRep,const char *nombSol){
289     fstream archLibrosBin(nombLibrosBin,ios::in | ios::out | ios::binary);
290     if (not archLibrosBin.is_open()){
291         cout<<"Error: el archivo "<<nombLibrosBin<<" no se pudo abrir"<<endl;
292         exit(1);
293     }
294     fstream archPrestamosBin(nombRegBin,ios::in | ios::out | ios::binary);
295     if (not archPrestamosBin.is_open()){
296         cout<<"Error: el archivo "<<nombRegBin<<" no se pudo abrir"<<endl;
297         exit(1);
298     }
299     ifstream archSolicitudes(nombSol,ios::in);
300     if (not archSolicitudes.is_open()){
301         cout<<"Error: el archivo "<<nombSol<<" no se pudo abrir"<<endl;
302         exit(1);
303     }
304     ofstream archReporte(nomRep,ios::out);
305     if (not archReporte.is_open()){
306         cout<<"Error: el archivo "<<nomRep<<" no se pudo abrir"<<endl;
307         exit(1);
308     }
309     int ddPrest,mmPrest,aaPrest,ddDev,mmDev,aaDev,codUsuario,fechaPrest,fechaDev;
310     char car,nombre[50],codLibro[8];
311     archSolicitudes>>ddPrest>>car>>mmPrest>>car>>aaPrest>>ddDev>>car>>mmDev>>
312             car>>aaDev;
313     fechaPrest=juntarFecha(ddPrest,mmPrest,aaPrest);
314     cout<<fechaPrest;
315     fechaDev=juntarFecha(ddDev,mmDev,aaDev);
316     archReporte<<setw(50)<<"BIBLIOTECA CENTRAL"<<setw(20)<<"Fecha:"<<
317             setfill('0')<<setw(2)<<ddPrest<<'/'<<setw(2)<<mmPrest<<'/'<<
318             setw(4)<<aaPrest<<endl<<setfill(' ');
319     archReporte<<"CODIGO"<<setw(15)<<"NOMBRE"<<setw(40)<<"COD. DE LIBRO"<<
320             setw(15)<<"TITULO"<<setw(30)<<"OBSERVACION"<<endl;
321     while(true){
322         archSolicitudes>>codUsuario;
323         if (archSolicitudes.eof()) break;
324         archSolicitudes.get();
325         archSolicitudes.getline(nombre,50,',');
326         archSolicitudes.getline(codLibro,8);
327         procesarPrestamo(archLibrosBin,archPrestamosBin,archReporte,
328                 codUsuario,nombre,codLibro,fechaPrest,fechaDev);
329     }
330 }
331 
332 int juntarFecha(int d,int m,int a){
333     return a*10000+m*100+d;
334 }
335 
336 void procesarPrestamo(fstream &archLibrosBin,fstream &archPrestamosBin,
337         ofstream &archReporte,int codUsuario, const char *nombre,
338         const char *codLibro,int fechaPrest,int fechaDev){
339     int posLibro=buscarLibro(archLibrosBin,codLibro);
340     int tamRegLibro=sizeof(struct Libros);
341     int tamRegPrestamo=sizeof(struct RegistroDePrestamos);
342     struct Libros libroDat;
343     if (posLibro!=NO_ENCONTRADO){
344         archLibrosBin.seekg(posLibro*tamRegLibro,ios::beg);
```

```
345          archLibrosBin.read(reinterpret_cast<char*>(&libroDat),tamRegLibro);
346          if (libroDat.cantidad>libroDat.prestamos){
347              int posPrestamo=buscarPrestamo(archPrestamosBin,codUsuario);
348              struct RegistroDePrestamos prestamoDat;
349              if (posPrestamo!=NO_ENCONTRADO){
350                  archPrestamosBin.seekg(posPrestamo*tamRegPrestamo,ios::beg);
351
                     archPrestamosBin.read(reinterpret_cast<char*>(&prestamoDat),tamRegPrestam
                     o);
352                  if (prestamoDat.cantidadDeLibros<7){
353                      if (verificarFechaDevoluciones(prestamoDat,fechaDev))
354                          actualizarPrestamo(archLibrosBin,archPrestamosBin,archReporte,
355                          prestamoDat,posPrestamo,libroDat,posLibro,fechaPrest);
356                      else
357
                             escribirReporte(archReporte,codUsuario,nombre,codLibro,libroDat.t
                             itulo,
358                              "Supero fecha limite");
359                  }
360                  else
361
                       escribirReporte(archReporte,codUsuario,nombre,codLibro,libroDat.titu
                       lo,
362                          "Supero el limite de prestamos");
363              }
364              else{
365                  prestamoDat.codigo=codUsuario;
366                  strcpy(prestamoDat.nombre,nombre);
367                  prestamoDat.cantidadDeLibros=0;
368                  actualizarPrestamo(archLibrosBin,archPrestamosBin,archReporte,
369                          prestamoDat,posPrestamo,libroDat,posLibro,fechaPrest);
370              }
371
372          }
373          else
374              escribirReporte(archReporte,codUsuario,nombre,codLibro,libroDat.titulo,
375                  "No hay libro disponible");
376
377      }
378      else
379          escribirReporte(archReporte,codUsuario,nombre,codLibro,"NN",
380                  "No existe el libro");
381  }
382
383  int buscarLibro(fstream &archLibrosBin, const char *codLibro){
384      int tamReg=sizeof(struct Libros);
385      int numReg=calcularNumRegistros(archLibrosBin,tamReg);
386      struct Libros libroDat;
387      for (int i=0;i<numReg;i++){
388          archLibrosBin.read(reinterpret_cast<char *>(&libroDat),tamReg);
389          if (strcmp(libroDat.codigo,codLibro)==0) return i;
390      }
391      return NO_ENCONTRADO;
392  }
393
394  int calcularNumRegistros(fstream &archBin,int tamReg){
395      archBin.seekg(0,ios::end);
396      int tamArch=archBin.tellg();
397      archBin.seekg(0,ios::beg);
398      return tamArch/tamReg;
399  }
400
401  void escribirReporte(ofstream &archReporte,int codUsuario, const char *nombre,
402          const char *codLibro,const char *tit,const char *obs){
403      archReporte<<setw(10)<<codUsuario<<' '<<left<<setw(50)<<nombre<<setw(10)<<
404              codLibro<<setw(50)<<tit<<setw(50)<<obs<<right<<endl;
405  }
406
407  int buscarPrestamo(fstream &archPrestamosBin,int codUsuario){
```

```
408       int tamReg=sizeof(struct RegistroDePrestamos);
409       int numReg=calcularNumRegistros(archPrestamosBin,tamReg);
410       struct RegistroDePrestamos registroDat;
411       for (int i=0;i<numReg;i++){
412           archPrestamosBin.read(reinterpret_cast<char *>(&registroDat),tamReg);
413           if (registroDat.codigo==codUsuario) return i;
414       }
415       return NO_ENCONTRADO;
416   }
417
418   void actualizarPrestamo(fstream &archLibrosBin,fstream &archPrestamosBin,
419           ofstream &archReporte,struct RegistroDePrestamos &prestamoDat,
420           int posPrestamo,struct Libros &libroDat,int posLibro,int fechaPrest){
421       int tamRegLibro=sizeof(struct Libros);
422       int tamRegPrestamo=sizeof(struct RegistroDePrestamos);
423
424       //actualizar el libro
425       libroDat.prestamos++;
426       archLibrosBin.seekg(posLibro*tamRegLibro,ios::beg);
427       archLibrosBin.write(reinterpret_cast<const char *>(&libroDat),tamRegLibro);
428       archLibrosBin.flush();
429
430       //actualizar el préstamo
431       strcpy(prestamoDat.libro[prestamoDat.cantidadDeLibros].codigo,libroDat.codigo);
432       prestamoDat.libro[prestamoDat.cantidadDeLibros].fecha=fechaPrest;
433       prestamoDat.cantidadDeLibros++;
434
435       if (posPrestamo==NO_ENCONTRADO)
436           archPrestamosBin.seekg(0,ios::end);
437       else
438           archPrestamosBin.seekg(posPrestamo*tamRegPrestamo,ios::beg);
439       archPrestamosBin.write(reinterpret_cast<const char *>(&prestamoDat),
440               tamRegPrestamo);
441       archPrestamosBin.flush();
442       escribirReporte(archReporte,prestamoDat.codigo,prestamoDat.nombre,
443               libroDat.codigo,libroDat.titulo,"Se le entrego el libro");
444   }
445
446   bool verificarFechaDevoluciones(struct RegistroDePrestamos &prestamoDat,
447           int fechaDev){
448       for (int i=0;i<prestamoDat.cantidadDeLibros;i++)
449           if (prestamoDat.libro[i].fecha<fechaDev)
450               return false;
451       return true;
452   }
453
```